

UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

REPORT DI PROGETTO

Ingegneria del Software: Tecniche Avanzate – Report del Progetto

Mario Celzo	Matricola: NF22500193
Renato Mancino	Matricola: NF22500242
Alfonso Maria Ferrara	Matricola: NF22500210

Repository: `https://github.com/mariocelzo/biblioflow-app`

Anno Accademico 2025-2026

Indice

1 Descrizione del Sistema **1**

1.1 Contesto Applicativo 1

1.2 La Piattaforma BiblioFlow 1

1.2.1 Analisi del Funzionamento 2

2 Struttura del Sistema **3**

2.1 Package Principali del Sistema 3

2.1.1 app (pagine) 3

2.1.2 app/api (route handler) 5

2.1.3 lib 5

2.1.4 components 7

2.1.5 accessibility e contexts 7

2.1.6 hooks e types 7

2.1.7 prisma 10

2.1.8 public (PWA) 10

3 Flusso di Funzionamento del Sistema **11**

3.1 Tipologie di Utilizzo 11

3.2 Flusso di Utilizzo 11

3.2.1 Flusso di Utilizzo da parte dello Studente 11

3.2.2 Flusso di Utilizzo da parte dello Staff 12

3.2.3 Flusso delle Automazioni Pianificate 12

Elenco delle figure

2.1 Analisi modulo app (pagine). 4
2.2 Analisi modulo app/api. 6
2.3 Analisi modulo lib. 8
2.4 Analisi modulo components. 9
2.5 Analisi modulo prisma (modello dei dati). 10

3.1 Sequence Diagram della prenotazione. 12
3.2 Sequence Diagram del check-in. 13
3.3 Sequence Diagram dello staff. 13
3.4 Sequence Diagram delle automazioni pianificate. 14

CAPITOLO 1

Descrizione del Sistema

1.1 Contesto Applicativo

Nelle biblioteche universitarie i posti studio sono una risorsa limitata e molto contesa, soprattutto nei periodi d'esame. In assenza di un sistema di prenotazione l'accesso segue la regola del primo arrivato: chi abita vicino alla sede occupa i posti nelle prime ore del mattino, mentre gli studenti pendolari, che dipendono dagli orari dei mezzi pubblici, arrivano quando la sala è già piena. È diffusa anche l'occupazione «informale» del posto, lasciando libri o zaino per ore, che rende la disponibilità reale diversa da quella apparente.

Il personale della biblioteca, d'altra parte, non dispone di dati affidabili sull'occupazione delle sale, sui posti che restano vuoti pur essendo stati promessi a qualcuno e sugli studenti che si prenotano senza presentarsi (*no-show*). Infine, gli studenti con esigenze di accessibilità hanno bisogno di sapere in anticipo quali posti sono raggiungibili e attrezzati.

La digitalizzazione della prenotazione dei posti, collegata a una conferma della presenza all'arrivo, permette di rendere l'accesso più equo e di liberare automaticamente i posti non utilizzati.

1.2 La Piattaforma BiblioFlow

BiblioFlow nasce in questo contesto come piattaforma web per la gestione dei posti studio e dei servizi di una biblioteca universitaria. È un'applicazione web progressiva (PWA) sviluppata con Next.js, React e TypeScript: si usa dal browser ma può essere installata su smartphone e continua a mostrare una pagina di cortesia in assenza di rete. I dati sono conservati in PostgreSQL e gestiti tramite l'ORM Prisma; l'autenticazione è affidata ad Auth.js (NextAuth), con accesso tramite email e password o account Google.

Il sistema distingue tre ruoli:

- **Studente:** prenota un posto scegliendolo sulla mappa della sala, conferma la presenza con il check-in, consulta il catalogo, chiede libri in prestito o la loro preparazione al banco e riceve notifiche;
- **Bibliotecario:** gestisce prenotazioni, posti, prestiti, richieste e anomalie, effettua il check-in degli studenti tramite lo scanner dei QR code e consulta le statistiche;
- **Amministratore:** ha gli stessi strumenti del bibliotecario e in più può attivare o disattivare gli account.

Due caratteristiche distinguono BiblioFlow da un semplice calendario di prenotazione: l'opzione **pendolare**, che aggiunge un margine di tolleranza sull'orario di arrivo per chi viaggia con i mezzi pubblici, e l'attenzione all'**accessibilità**, con posti accessibili segnalati sulla

mappa, preferenze di visualizzazione (alto contrasto, testo grande, movimento ridotto) e navigazione completa da tastiera.

1.2.1 Analisi del Funzionamento

Il funzionamento di BiblioFlow ruota attorno al ciclo di vita della prenotazione di un posto e può essere descritto attraverso i seguenti passaggi:

- **Autenticazione:** lo studente accede con email e password (verificata con bcrypt) o con Google. La sessione è un token JWT conservato in un cookie; un middleware intercetta le richieste e respinge quelle prive del cookie di sessione, mentre i controlli sul ruolo sono svolti dalle singole route.
- **Prenotazione guidata:** una procedura in quattro passi raccoglie data, durata (due ore, mezza giornata di mattina o di pomeriggio, giornata intera), sala e posto. La scelta del posto avviene su una mappa interattiva che mostra lo stato di ogni posto e le sue caratteristiche (presa elettrica, accessibilità). Il server verifica gli orari di apertura della sala e l'assenza di sovrapposizioni prima di creare la prenotazione nello stato `CONFERMATA`.
- **Check-in e uso del posto:** all'arrivo lo studente conferma la presenza dalla home, dove la prenotazione del giorno è accompagnata da un QR code, oppure si fa scansionare il QR dal personale. Il check-in è ammesso in una finestra di quindici minuti attorno all'inizio della prenotazione e porta la prenotazione in `CHECK_IN` e il posto in `OCCUPATO`; al termine la prenotazione diventa `COMPLETATA`.
- **Automazioni:** un'attività pianificata invia i promemoria di check-in, avvisa delle scadenze dei prestiti e rilascia i posti delle prenotazioni non confermate entro il margine (stato `NO_SHOW`).
- **Notifiche e tracciabilità:** ogni operazione rilevante produce una notifica persistente per l'utente e un evento nel registro `LogEvento`, che alimenta la dashboard e le statistiche dello staff.

CAPITOLO 2

Struttura del Sistema

L'architettura di BiblioFlow segue la struttura di un'applicazione Next.js con App Router: pagine e route handler convivono nella cartella `src/app`, mentre la logica condivisa, i componenti dell'interfaccia e il modello dei dati sono organizzati in package distinti, ognuno preposto a una specifica responsabilità. Di seguito viene presentata l'analisi dei moduli principali, così come si presentano nella versione di partenza del progetto.

2.1 Package Principali del Sistema

2.1.1 app (pagine)

L'obiettivo è fornire le schermate dell'applicazione. Ogni cartella sotto `src/app` corrisponde a un percorso dell'interfaccia; le pagine dello studente sono componenti client che dialogano con le API tramite `fetch`, mentre la dashboard dello staff è resa lato server e legge i dati direttamente con Prisma.

Moduli principali:

- `layout.tsx`: struttura comune a tutte le pagine; monta i provider di sessione, tema, accessibilità e PWA, l'intestazione e i componenti di accessibilità (salto al contenuto, annunci per i lettori di schermo).
- `page.tsx`: home dello studente; mostra la prenotazione attiva con il QR code per il check-in e le scorciatoie verso le funzioni principali.
- `login`, `registrazione`, `recupera-password`, `reset-password`: accesso e gestione delle credenziali.
- `prenota`: procedura guidata di prenotazione in quattro passi (data, durata, sala, posto) con mappa interattiva e opzione pendolare.
- `prenotazioni` e `prenotazioni/[id]/estendi`: elenco delle prenotazioni dello studente, cancellazione ed estensione dell'orario.
- `libri`, `libri/[id]`, `prestiti`: catalogo, dettaglio di un libro, richiesta e rinnovo dei prestiti.
- `notifiche`, `profilo`: centro notifiche e dati del profilo, comprese le preferenze di pendolarismo e accessibilità.
- `admin/*`: area dello staff, con dashboard, gestione di prenotazioni, posti, prestiti, richieste di preparazione, utenti e anomalie, scanner dei QR code e statistiche.

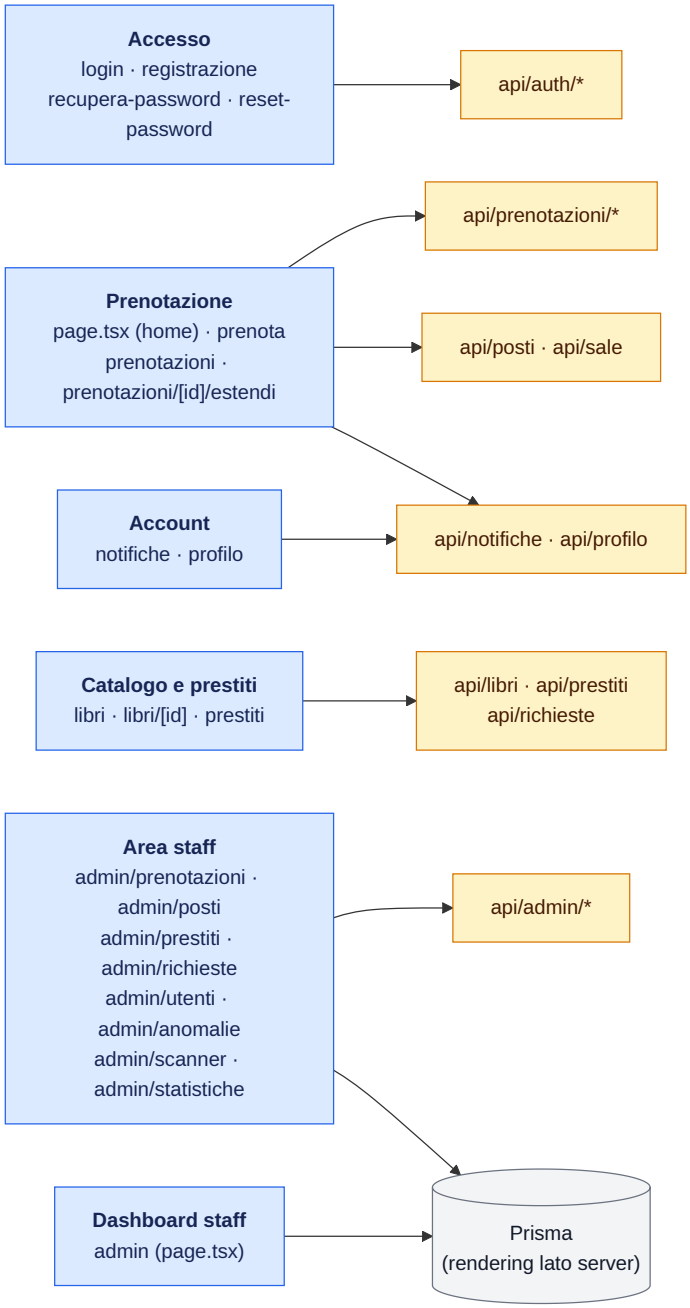


Figura 2.1: Analisi modulo app (pagine).

2.1.2 app/api (route handler)

Questo package rappresenta il backend dell'applicazione. Ogni file `route.ts` espone uno o più metodi HTTP e restituisce JSON; le route validano l'input, verificano la sessione dove previsto e accedono ai dati tramite Prisma.

Moduli principali:

- `auth/*`: gestore di Auth.js e flussi di registrazione, verifica dell'email, recupero e reimpostazione della password, con limitazione della frequenza delle richieste.
- `prenotazioni`, `prenotazioni/[id]`, `prenotazioni/[id]/check-in`, `prenotazioni/[id]/estendi`: elenco, creazione, lettura, cancellazione, check-in ed estensione delle prenotazioni.
- `posti`, `sale`, `disponibilita-giorni`: stato dei posti, sale con i relativi orari e giorni prenotabili.
- `libri`, `prestiti`, `richieste`: catalogo, prestiti e rinnovi, richieste di preparazione dei libri al banco.
- `notifiche`, `profilo`: lettura, marcatura ed eliminazione delle notifiche; lettura e modifica del profilo.
- `admin/*`: operazioni dello staff (annullamento e modifica delle prenotazioni, stato dei posti, prestiti, richieste, utenti, anomalie, statistiche, validazione dei QR code).
- `cron/automations`: punto di ingresso dell'attività pianificata, protetto da un segreto condiviso.
- `sse/posti`: canale Server-Sent Events per gli aggiornamenti in tempo reale dei posti.

A monte delle route opera `src/middleware.ts`, che lascia passare le pagine e le API pubbliche (accesso, registrazione, `api/auth`, `api/cron`) e per tutte le altre verifica la presenza del cookie di sessione: in sua assenza risponde 401 alle API e reindirizza le pagine al login, conservando la destinazione richiesta.

2.1.3 lib

Raccoglie i servizi condivisi dal backend: autenticazione, accesso ai dati, automazioni e utilità trasversali.

Moduli principali:

- `auth.ts`: configurazione di Auth.js con i provider Credentials e Google, i callback che arricchiscono il token con ruolo e preferenze dell'utente e le funzioni `hasRole` e `isStaff`.
- `prisma.ts`: istanza unica del client Prisma collegata a PostgreSQL tramite l'adattatore `pg`.
- `automation-service.ts`: logica delle automazioni periodiche: `sendCheckInReminders`, `sendLoanExpiryAlerts`, `releaseNoShowReservations`, `notifyAvailableSeat` e l'orchestratore `runAllAutomations`.
- `realtime-events.ts` e `sse-emitter.ts`: emettitore di eventi in memoria e funzioni di pubblicazione degli aggiornamenti dei posti e delle notifiche sul canale SSE.

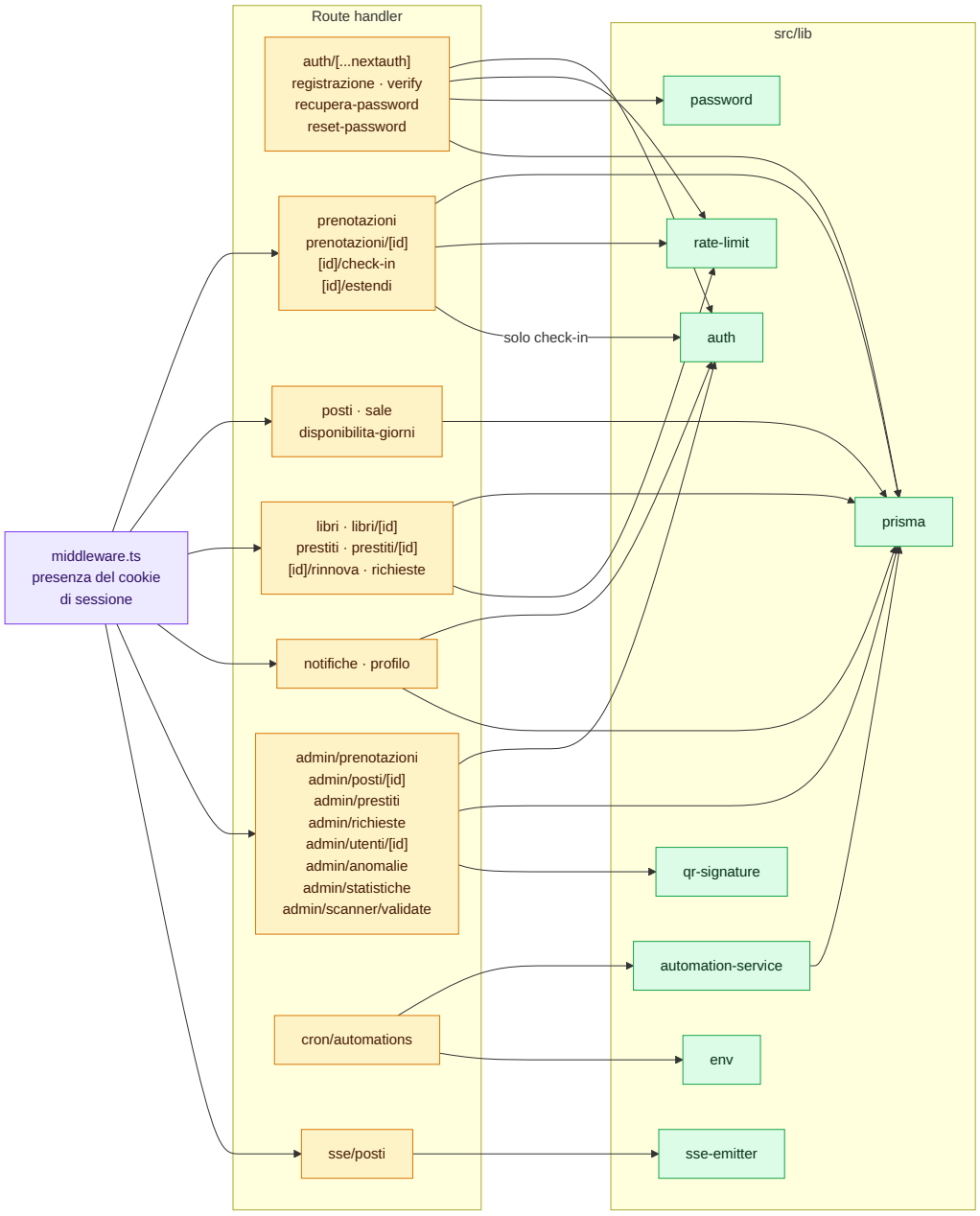


Figura 2.2: Analisi modulo app/api.

- `qr-signature.ts`: firma HMAC e verifica dei QR code usati per il check-in, con controllo della scadenza.
- `rate-limit.ts`: limitatori di frequenza per indirizzo IP (accesso, registrazione, prenotazioni, prestiti, API).
- `password.ts`, `env.ts`, `utils.ts`: hashing e regole delle password, validazione delle variabili d'ambiente all'avvio, composizione delle classi CSS.

2.1.4 components

Contiene i componenti React riutilizzabili che compongono l'interfaccia. Questo approccio modulare facilita la manutenzione e la coerenza visiva tra l'area dello studente e quella dello staff.

Moduli principali:

- `mappa-biblioteca.tsx`: mappa interattiva dei posti di una sala, con legenda degli stati e delle caratteristiche; sotto i 768 pixel di larghezza cede il posto a `mobile-posti-grid.tsx`, una griglia pensata per lo schermo dello smartphone.
- `qrcode-checkin.tsx`: mostra il QR code della prenotazione e invia la conferma di check-in.
- `admin/*`: menu di azioni su prenotazioni, posti, prestiti, utenti e anomalie, filtri, card della dashboard e grafici delle statistiche.
- `layout/*` e `providers/*`: intestazione con il contatore delle notifiche, barra laterale dello staff, provider di sessione, tema e PWA.
- `ui/*`: primitive grafiche (pulsanti, card, dialog, menu, schede, selettori) costruite su Radix UI.

2.1.5 accessibility e contexts

Questo package raccoglie il supporto all'accessibilità, che nel progetto è un requisito e non un'aggiunta.

- `contexts/accessibility-context.tsx`: stato globale delle preferenze (alto contrasto, testo grande, dimensione del carattere, movimento ridotto, tema scuro, navigazione da tastiera), salvate nel browser e applicate all'intera interfaccia.
- `components/accessibility/*`: collegamento per saltare al contenuto, scorciatoie da tastiera, intrappolamento del focus nei dialog e annunci per i lettori di schermo (`LiveAnnouncer`).

2.1.6 hooks e types

- `hooks/use-sse.ts`: hook React che apre una connessione `EventSource`, gestisce la riconnessione e consegna gli eventi ricevuti; `usePostiRealtime` lo specializza per il canale dei posti.
- `types/index.ts`: tipi TypeScript condivisi tra pagine e componenti.



Figura 2.3: Analisi modulo lib.

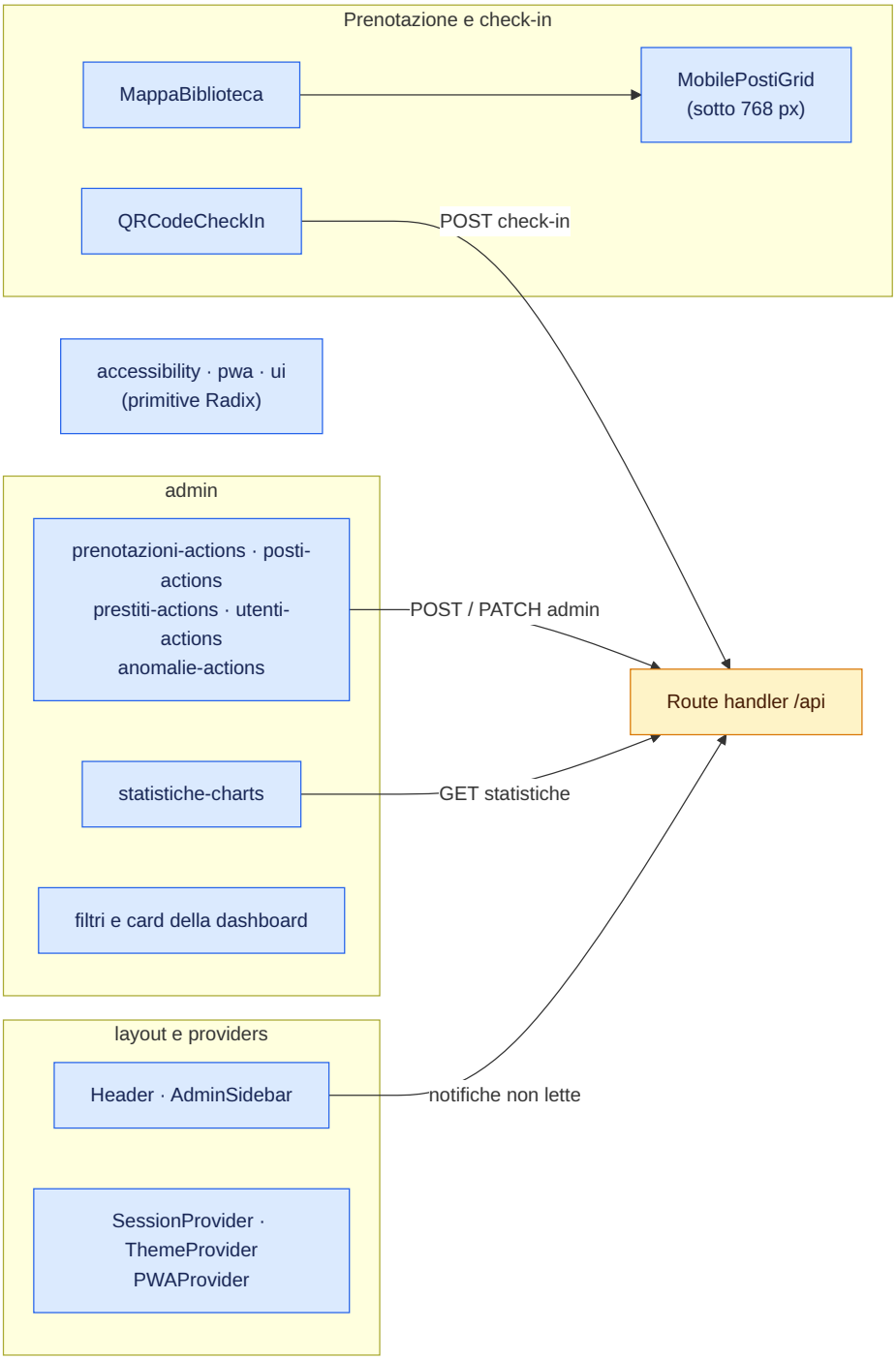


Figura 2.4: Analisi modulo components.

2.1.7 prisma

Questa cartella contiene il modello dei dati e gli strumenti per crearlo.

- `schema.prisma`: definisce undici entità, tra cui utenti, sale, posti, prenotazioni, libri, prestiti, notifiche e registro degli eventi, insieme agli stati di posti, prenotazioni e prestiti. La data di una prenotazione è salvata come `@db.Date` e i suoi orari come `@db.Time`.
- `migrations/*`: storico delle migrazioni del database.
- `seed.ts`: popola un'installazione nuova con sale, posti, libri e account dimostrativi per ciascun ruolo.

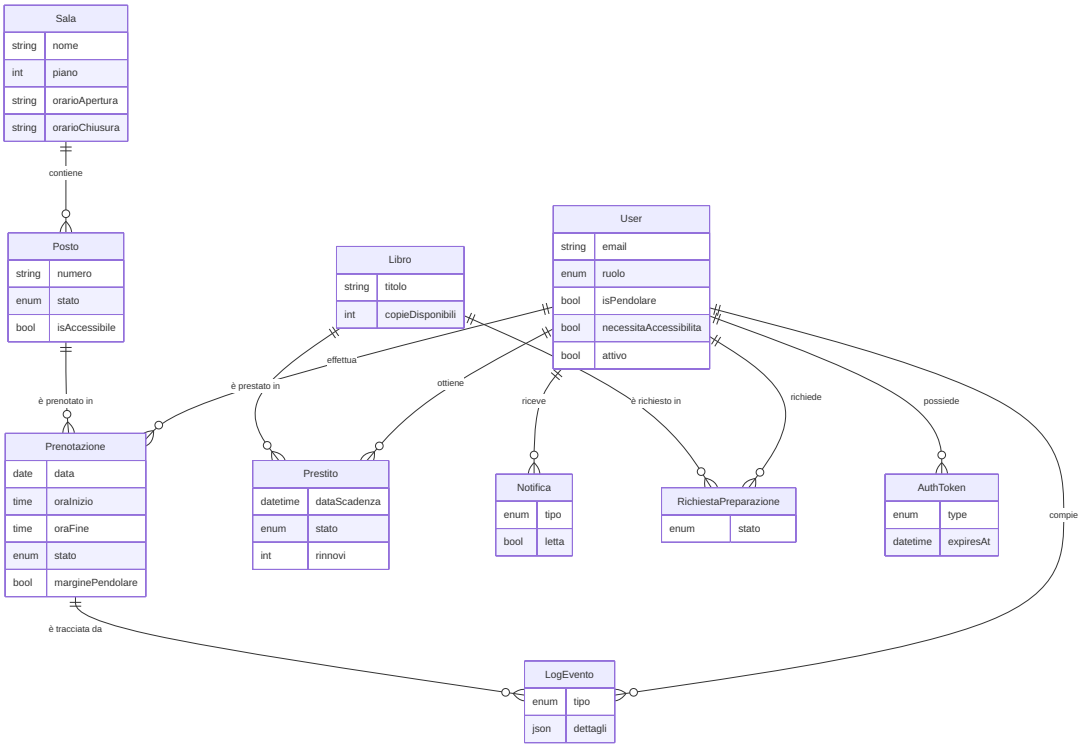


Figura 2.5: Analisi modulo prisma (modello dei dati).

2.1.8 public (PWA)

Contiene le risorse statiche che rendono BiblioFlow installabile e utilizzabile anche con una connessione instabile.

- `manifest.json`: nome, icone, colori e scorciatoie dell'applicazione installata.
- `sw.js`: service worker con strategia *network first* per le pagine e le API e *cache first* per le risorse statiche; senza rete mostra `offline.html`.
- `components/pwa/*`: banner di installazione e indicatore dello stato offline.

CAPITOLO 3

Flusso di Funzionamento del Sistema

3.1 Tipologie di Utilizzo

BiblioFlow è stato progettato per accompagnare la giornata in biblioteca dal punto di vista di chi studia, di chi gestisce la sala e del sistema stesso, che interviene in autonomia a intervalli regolari. Il sistema offre quindi tre modalità principali di utilizzo:

- **Utilizzo da parte dello studente:** tramite browser o applicazione installata, lo studente prenota il posto, riceve i promemoria e conferma la presenza con il check-in.
- **Utilizzo da parte dello staff:** bibliotecari e amministratori usano l'area `/admin` per effettuare il check-in con lo scanner, annullare o modificare prenotazioni e risolvere le anomalie.
- **Automazioni pianificate:** un'attività pianificata invoca periodicamente l'endpoint `/api/cron/automations`, che esegue promemoria, avvisi e rilascio dei posti non occupati.

3.2 Flusso di Utilizzo

In questa sezione vengono dettagliati i flussi operativi che caratterizzano le tre modalità di utilizzo. Per ognuna si descrive la sequenza di eventi che porta dall'azione dell'utente, o dell'attività pianificata, al risultato finale.

3.2.1 Flusso di Utilizzo da parte dello Studente

Prenotazione.

1. Lo studente apre `/prenota` e sceglie la data, la durata e la sala; la pagina carica sale e posti con `GET/api/sale` e `GET/api/posti`.
2. Sulla mappa sceglie un posto libero, eventualmente attiva il margine pendolare, e conferma.
3. La pagina invia `POST/api/prenotazioni` con identificativo dell'utente, posto, data e orari.
4. Il route handler controlla i campi obbligatori, legge utente, posto e sala, verifica che l'intervallo rientri nell'orario di apertura e che non esistano prenotazioni attive sovrapposte sul posto o per l'utente.
5. Se i controlli sono superati crea la prenotazione nello stato `CONFERMATA`, registra l'evento `PRENOTAZIONE_CREATA`, invia una notifica e risponde 201; in caso di sovrapposizione risponde 409.

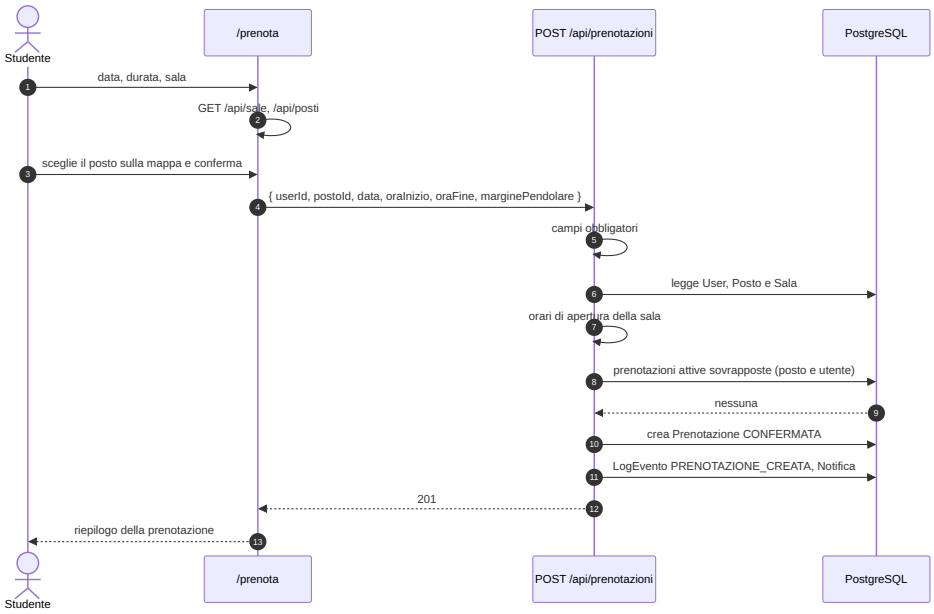


Figura 3.1: Sequence Diagram della prenotazione.

Check-in.

1. Nel giorno della prenotazione la home mostra il QR code della prenotazione e il pulsante di conferma della presenza.
2. La conferma invia `POST/api/prenotazioni/[id]/check-in`; il route handler verifica la sessione e che la prenotazione appartenga all'utente.
3. Se il momento della richiesta cade nella finestra di quindici minuti attorno all'inizio, la prenotazione passa a `CHECK_IN` e il posto a `OCCUPATO`; altrimenti la richiesta è rifiutata come scaduta.

3.2.2 Flusso di Utilizzo da parte dello Staff

1. Il bibliotecario apre `/admin/scanner` e inquadra il QR code mostrato dallo studente.
2. La pagina invia il contenuto del QR a `POST/api/admin/scanner/validate`, che verifica sessione e ruolo, la firma e la validità temporale del codice.
3. Il route handler controlla che la prenotazione esista, appartenga allo studente indicato nel QR e non sia annullata, scaduta o già in corso; quindi registra il check-in, occupa il posto, crea l'evento e la notifica e mostra allo staff i dati dello studente.
4. Dalla pagina `/admin/prenotazioni` lo staff può annullare una prenotazione (`POST/api/admin/prenotazioni`, azione `ANNULLA_SINGOLA`): la prenotazione diventa `CANCELLATA`, il posto torna disponibile se era occupato e lo studente riceve una notifica.

3.2.3 Flusso delle Automazioni Pianificate

1. L'attività pianificata di Vercel (Vercel Cron) chiama `GET/api/cron/automations`, passando il segreto `CRON_SECRET` nell'intestazione di autorizzazione.

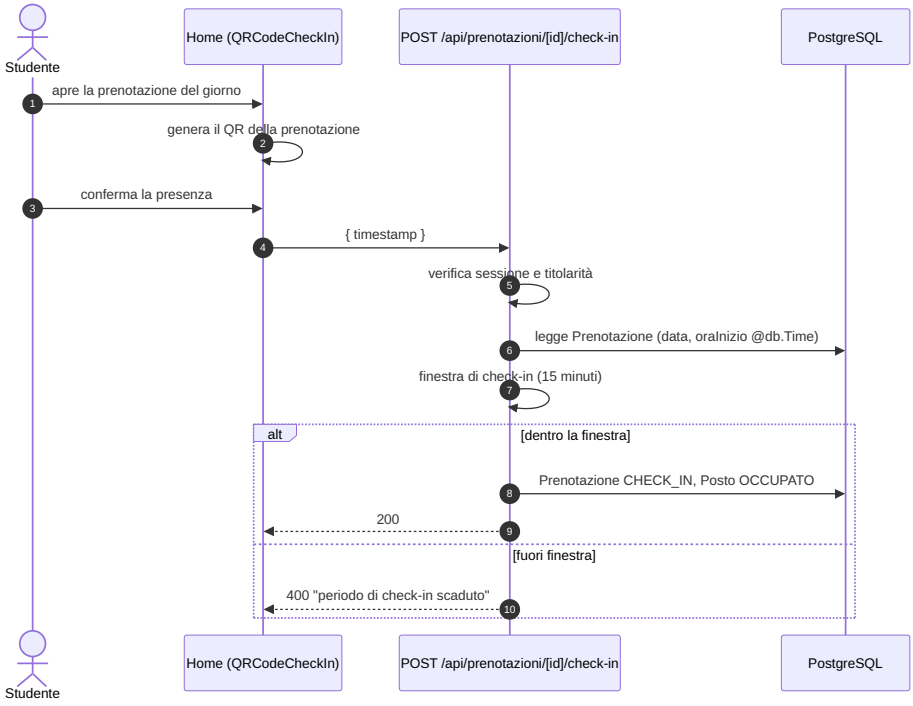


Figura 3.2: Sequence Diagram del check-in.

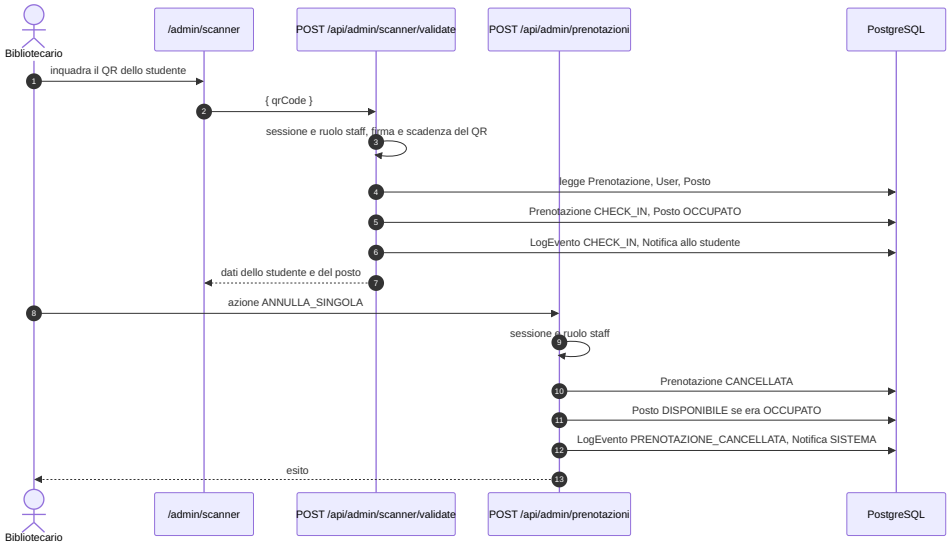


Figura 3.3: Sequence Diagram dello staff.

2. Il route handler confronta il token con il segreto configurato e invoca `runAllAutomations()`.
3. Il servizio invia i promemoria di check-in alle prenotazioni che iniziano entro venti minuti, gli avvisi per i prestiti in scadenza e rilascia le prenotazioni `CONFERMATA` non confermate oltre il margine: stato `NO_SHOW`, posto `DISPONIBILE`, notifica all'utente ed evento `NO_SHOW_AUTO`.
4. Al termine restituisce un riepilogo delle operazioni eseguite.

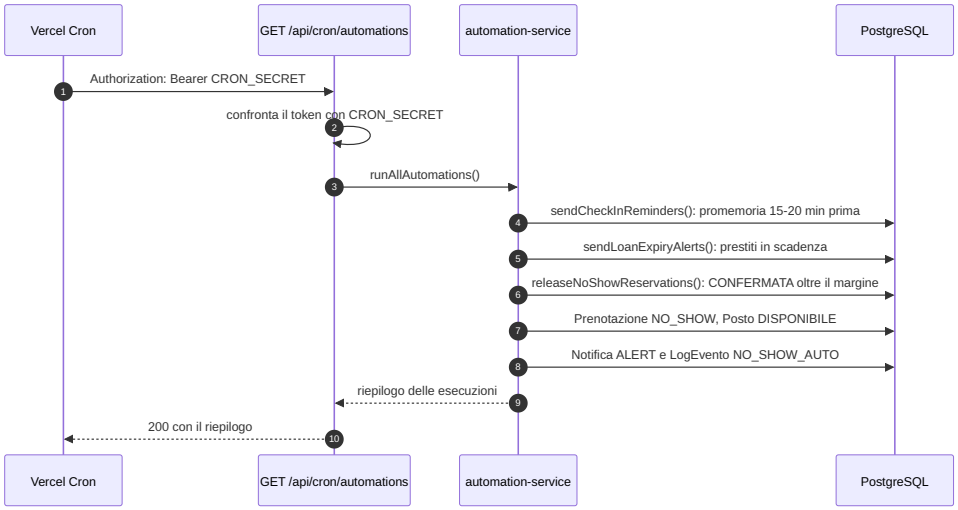


Figura 3.4: Sequence Diagram delle automazioni pianificate.